

Experiment ‘p_6_9_arrays_equal_array’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_9_arrays_equal_array

Natural language statement: Write a method that checks whether two arrays have the same elements in the same order.

Method signature: p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal: bool)

Ensures

- are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.Length ==> first[i] == second[i])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that checks whether two arrays have the same elements in the same order.

The signature should be:

`p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal: bool)`

The method should respect the following contract:

`ensures are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.Length
 ↪ ==> first[i] == second[i])`

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn1jD2AcgOc4scarbytaidXmSYxi4

Model: gpt-5.5-2026-04-23

Created at: 1780577087

Finish reason: stop

Usage: 320 tokens in, and 3087 tokens out

```
{
  if first.Length != second.Length {
    are_equal := false;
    assert !(first.Length == second.Length);
    assert !(first.Length == second.Length && (forall i :: 0 <= i < first.Length ==> first
      ↪ [i] == second[i]));
  } else {
    var i := 0;
    are_equal := true;
    while i < first.Length
      invariant first.Length == second.Length
      invariant 0 <= i <= first.Length
      invariant are_equal ==> (forall k :: 0 <= k < i ==> first[k] == second[k])
      invariant !are_equal ==> (exists k :: 0 <= k < i && first[k] != second[k])
      decreases first.Length - i
    {
      if first[i] != second[i] {
        are_equal := false;
        assert 0 <= i < i + 1 && first[i] != second[i];
        assert exists k :: 0 <= k < i + 1 && first[k] != second[k];
      }
    }
  }
}
```

```

    } else {
      assert first[i] == second[i];
      if are_equal {
        assert forall k :: 0 <= k < i ==> first[k] == second[k];
        forall k | 0 <= k < i + 1
          ensures first[k] == second[k]
        {
          if k < i {
            assert first[k] == second[k];
          } else {
            assert k == i;
          }
        }
      } else {
        ghost var k :| 0 <= k < i && first[k] != second[k];
        assert 0 <= k < i + 1 && first[k] != second[k];
        assert exists j :: 0 <= j < i + 1 && first[j] != second[j];
      }
    }
    i := i + 1;
  }

  assert i == first.Length;
  if are_equal {
    assert forall k :: 0 <= k < first.Length ==> first[k] == second[k];
  } else {
    ghost var k :| 0 <= k < first.Length && first[k] != second[k];
    if forall j :: 0 <= j < first.Length ==> first[j] == second[j] {
      assert first[k] == second[k];
      assert false;
    }
    assert !(forall j :: 0 <= j < first.Length ==> first[j] == second[j]);
  }
}

```

Final Program

```

method p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal
  ↪ : bool)
  ensures are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.
    ↪ Length ==> first[i] == second[i])
{
  if first.Length != second.Length {
    are_equal := false;
    assert !(first.Length == second.Length);
    assert !(first.Length == second.Length && (forall i :: 0 <= i < first.Length ==> first
      ↪ [i] == second[i]));
  } else {
    var i := 0;

```

```

are_equal := true;
while i < first.Length
  invariant first.Length == second.Length
  invariant 0 <= i <= first.Length
  invariant are_equal ==> (forall k :: 0 <= k < i ==> first[k] == second[k])
  invariant !are_equal ==> (exists k :: 0 <= k < i && first[k] != second[k])
  decreases first.Length - i
{
  if first[i] != second[i] {
    are_equal := false;
    assert 0 <= i < i + 1 && first[i] != second[i];
    assert exists k :: 0 <= k < i + 1 && first[k] != second[k];
  } else {
    assert first[i] == second[i];
    if are_equal {
      assert forall k :: 0 <= k < i ==> first[k] == second[k];
      forall k | 0 <= k < i + 1
        ensures first[k] == second[k]
      {
        if k < i {
          assert first[k] == second[k];
        } else {
          assert k == i;
        }
      }
    } else {
      ghost var k :: 0 <= k < i && first[k] != second[k];
      assert 0 <= k < i + 1 && first[k] != second[k];
      assert exists j :: 0 <= j < i + 1 && first[j] != second[j];
    }
  }
  i := i + 1;
}

assert i == first.Length;
if are_equal {
  assert forall k :: 0 <= k < first.Length ==> first[k] == second[k];
} else {
  ghost var k :: 0 <= k < first.Length && first[k] != second[k];
  if forall j :: 0 <= j < first.Length ==> first[j] == second[j] {
    assert first[k] == second[k];
    assert false;
  }
  assert !(forall j :: 0 <= j < first.Length ==> first[j] == second[j]);
}
}
}

```

Total Token Usage

Input tokens: 320

Output tokens: 3087

Reasoning tokens: 2486

Sum of ‘total tokens’: 3407

Experiment Timings

Overall Experiment started at 1780577084225, ended at 1780577168851, lasting 84626ms (84.63 seconds)

Iteration #1 started at 1780577084229, ended at 1780577168851, lasting 84622ms (84.62 seconds)